SENTIMENT ANALYSIS OF APP REVIEWS USING MACHINE LEARNING

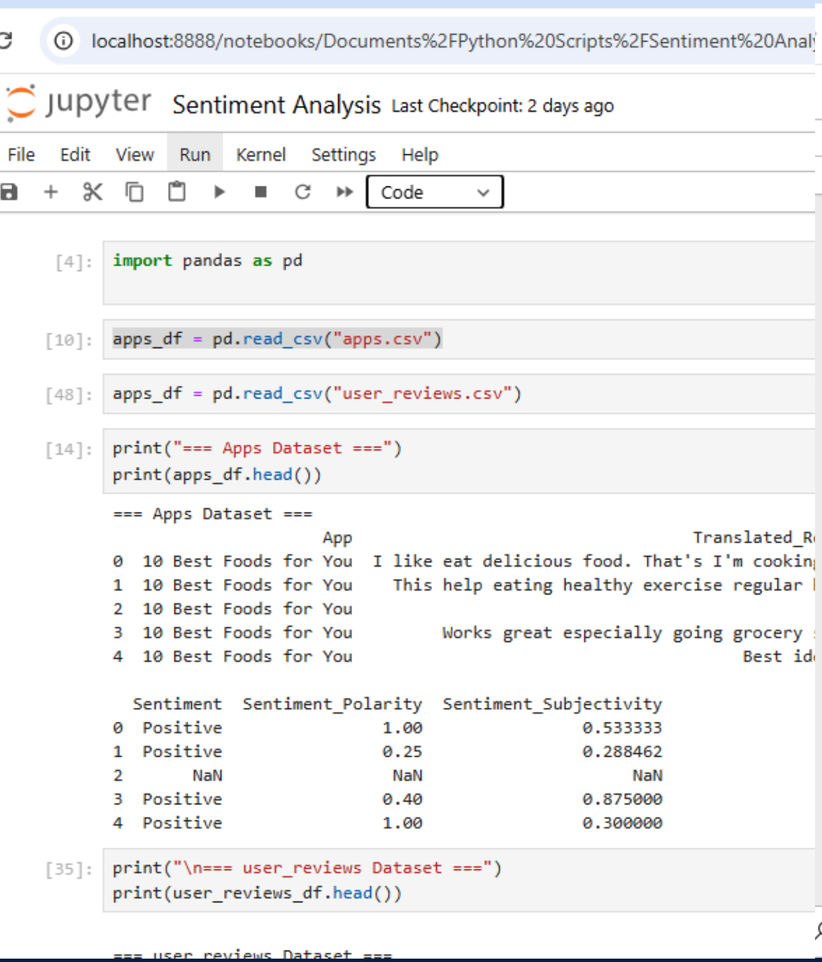
Presented by: Ajayi Oluwaseyi

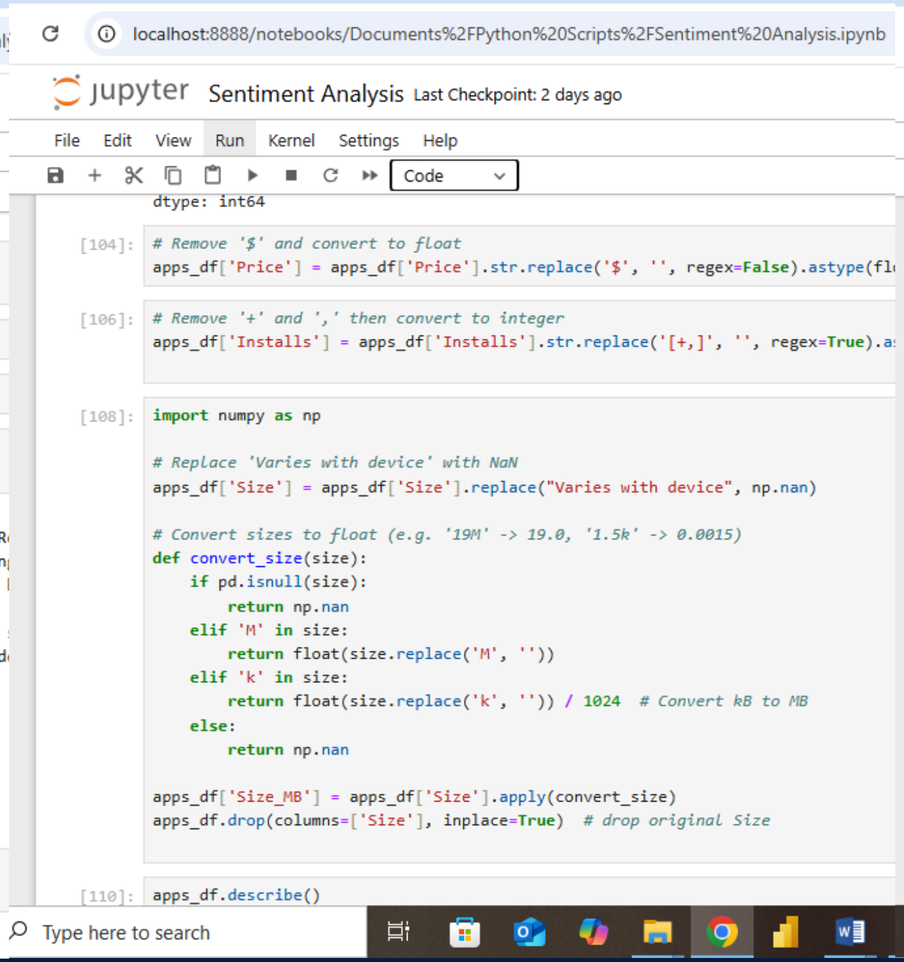
What Are Users Really Saying ?

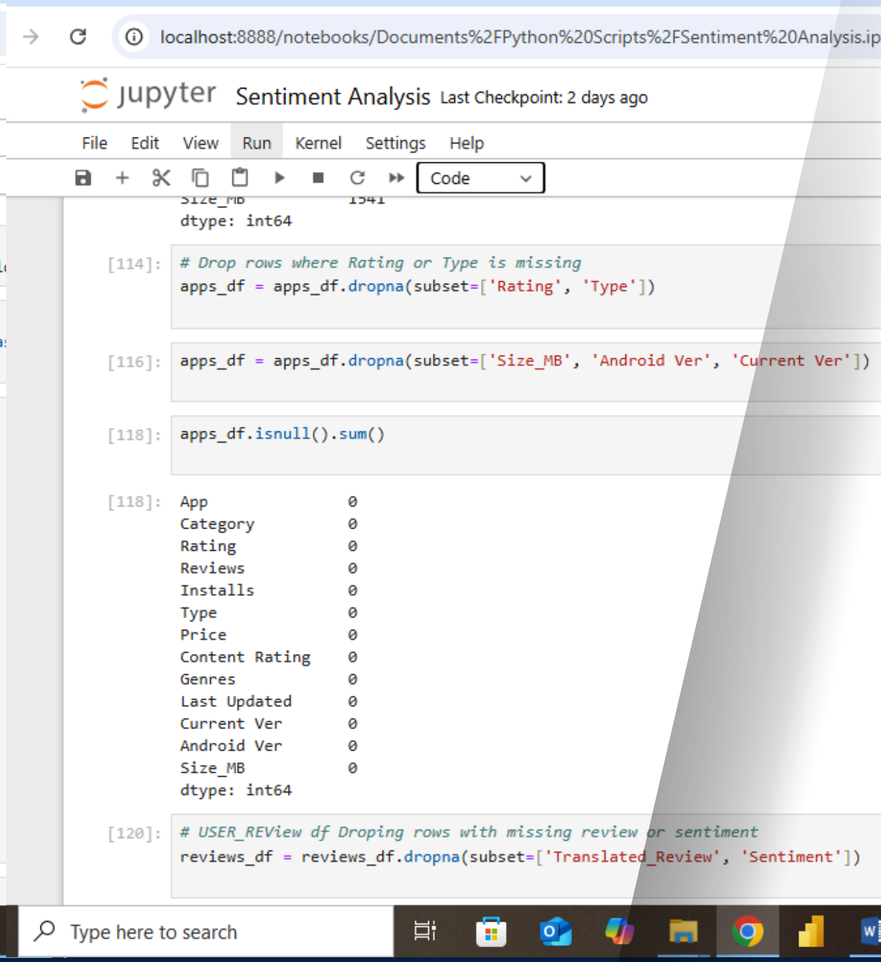
With mobile apps dominating our digital lives, understanding how users feel about them is key to improving their quality and customer experience. The objective was clear:

"Build a sentiment analysis model that accurately classifies user reviews as Positive, Neutral, or Negative."

But more than just building a model, this project was about unlocking insights — from emotions hidden in reviews, to what app categories generate the most love or frustration, and how ratings align with user sentiment.





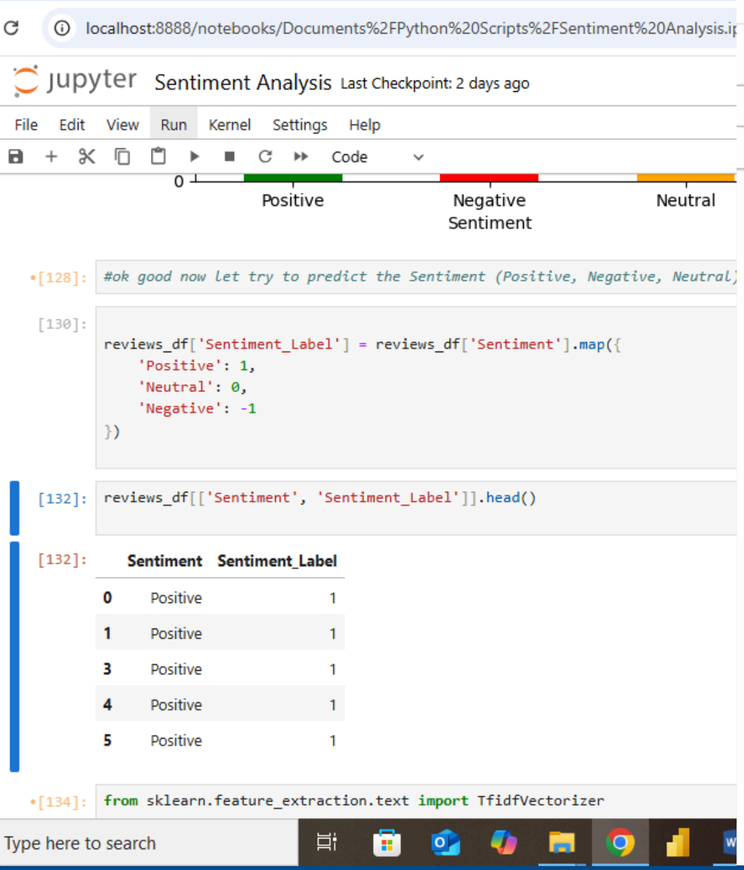


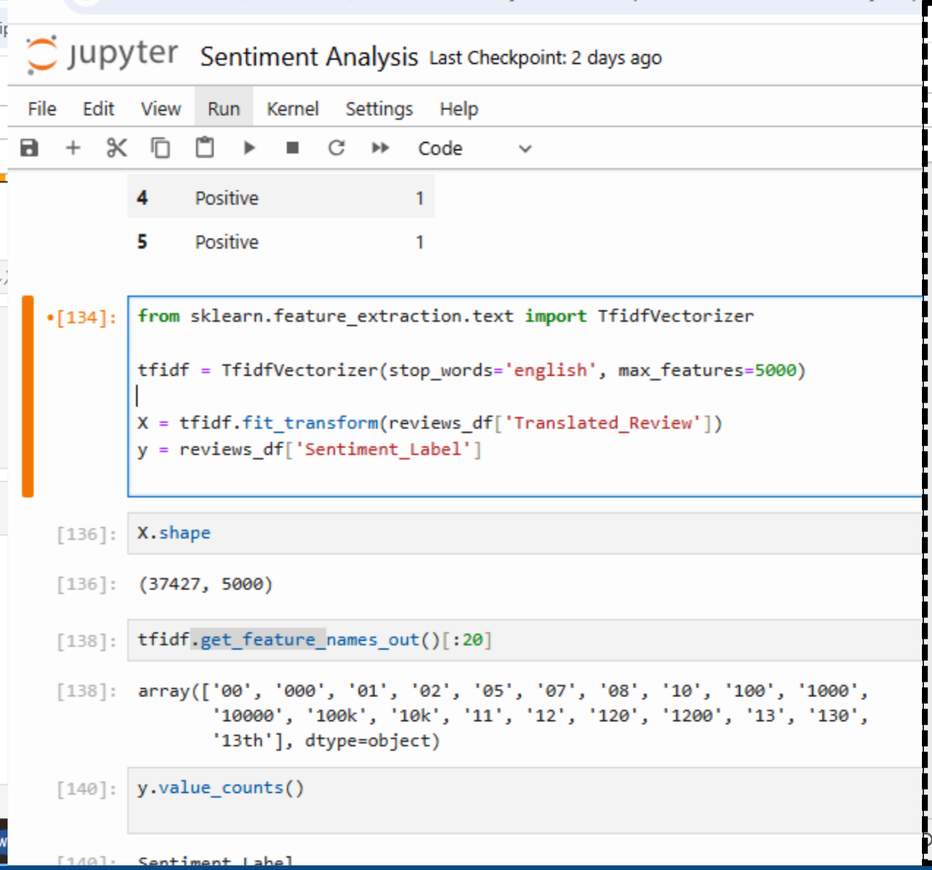
For this Project we will be using two datasets:

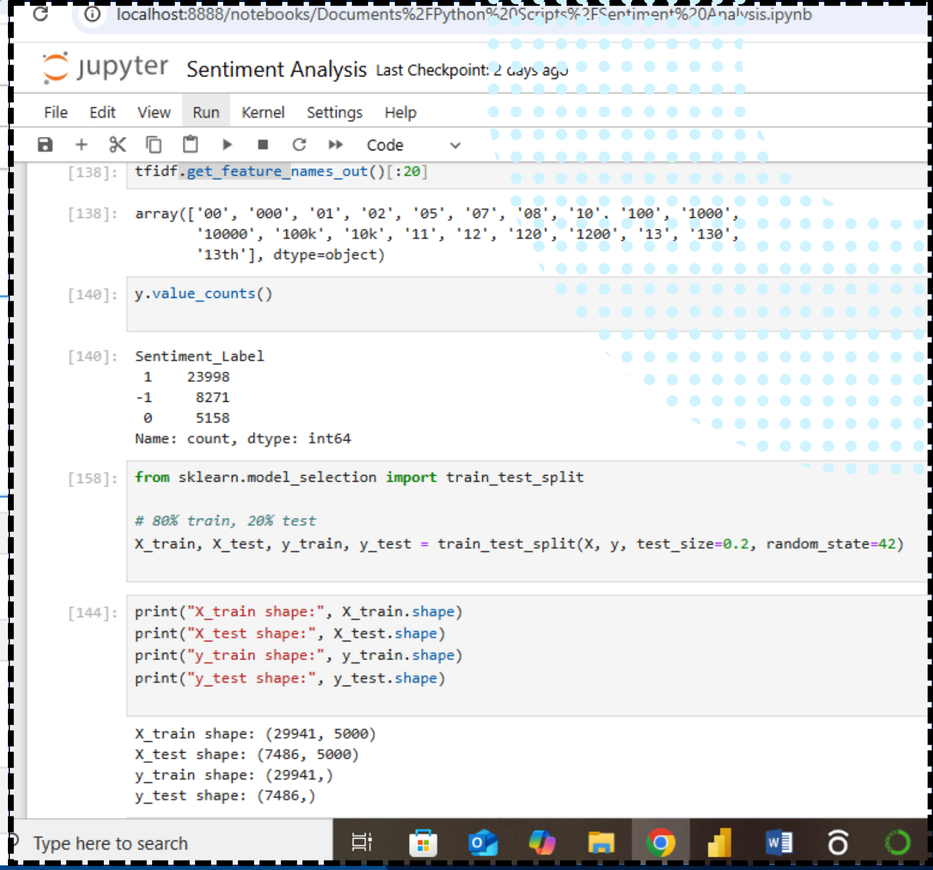
“Apps_df” : Containing metadata like App Name, Category, Rating, Type (Free or Paid).
“Reviews_df”: Containing user reviews and the actual text to analyze.

We started by:

- Cleaning the text: Removed punctuation, transformed to lowercase, and stripped out irrelevant symbols.
- Labeling the sentiments: The dataset had a Sentiment column already tagged, saving us from manual annotation.







Teaching the Machine to Read

Text can’t be fed directly into machine learning models — it has to be transformed.

So, I used TF-IDF Vectorization to convert words into numbers that capture importance in context. This allowed us to represent each review in a way the model could understand without being overwhelmed by common or unimportant words.

jupyter Sentiment Analysis Last Checkpoint: 2 days ago

File Edit View Run Kernel Settings Help

Code

[146]:
```
from sklearn.naive_bayes import MultinomialNB

# this Create the model
model = MultinomialNB()

model.fit(X_train, y_train)
```

[146]:
```
MultinomialNB()
```

[150]:
```
from sklearn.metrics import accuracy_score

y_pred = model.predict(X_test)

# Check accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", round(accuracy * 100, 2), "%")
```

```
Model Accuracy: 74.19 %
```

[152]:
```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred, labels=[1, 0, -1])

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Positive", "Neutral", "Negative"])
```

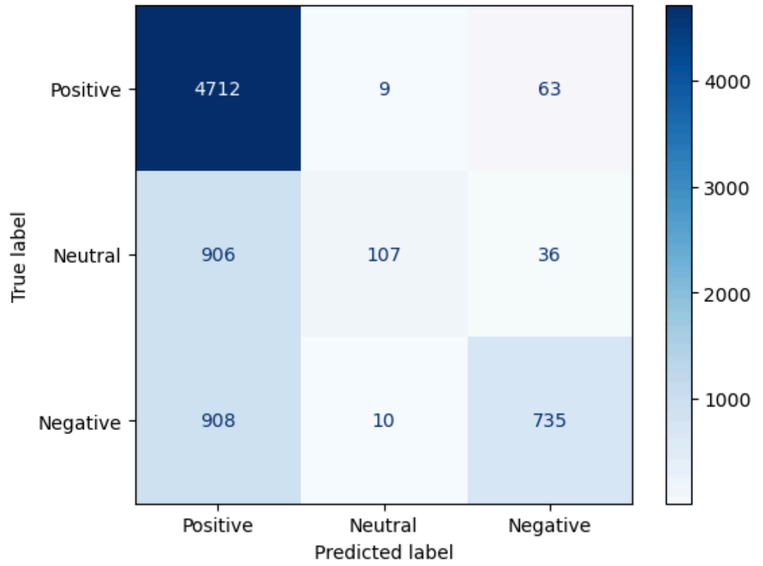
upyter Sentiment Analysis Last Checkpoint: 2 days ago

Edit View Run Kernel Settings Help

Code JupyterLab

```
disp.plot(cmap='Blues')
```

2]:
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x28c9f1a2f30>
```



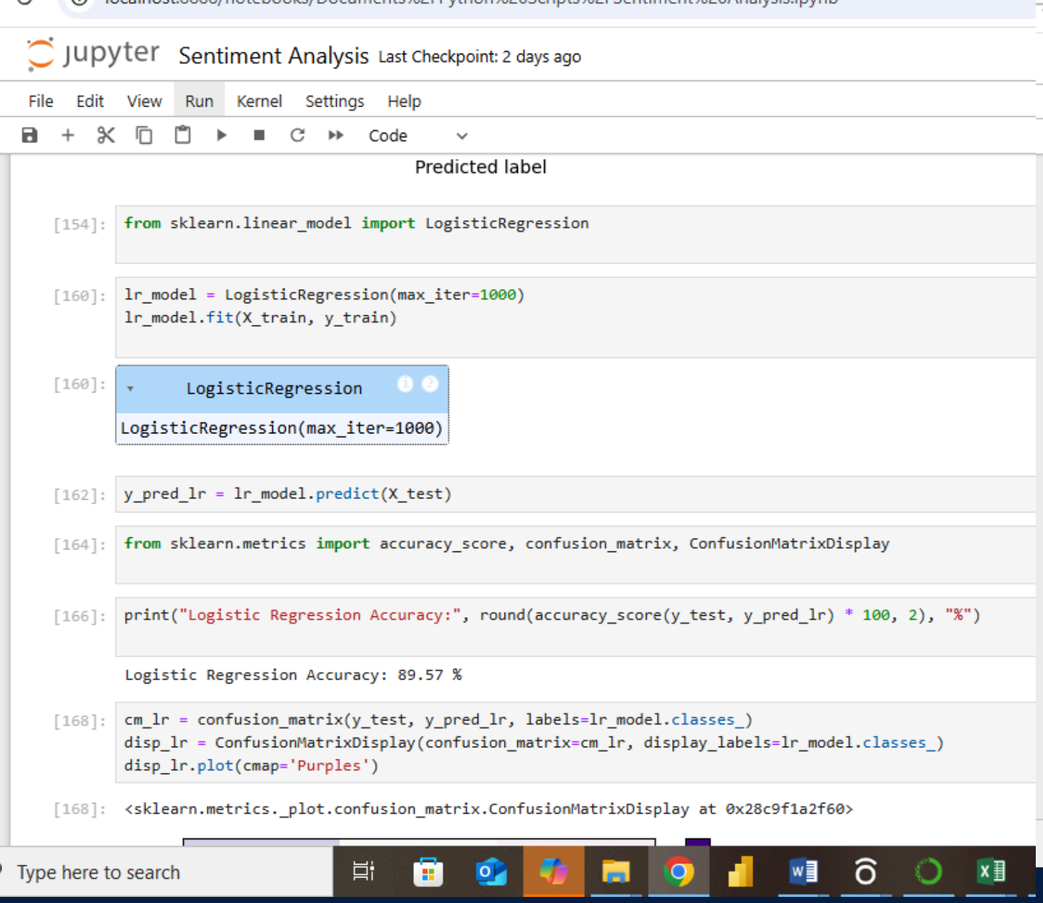
4]:
```
from sklearn.linear_model import LogisticRegression
```

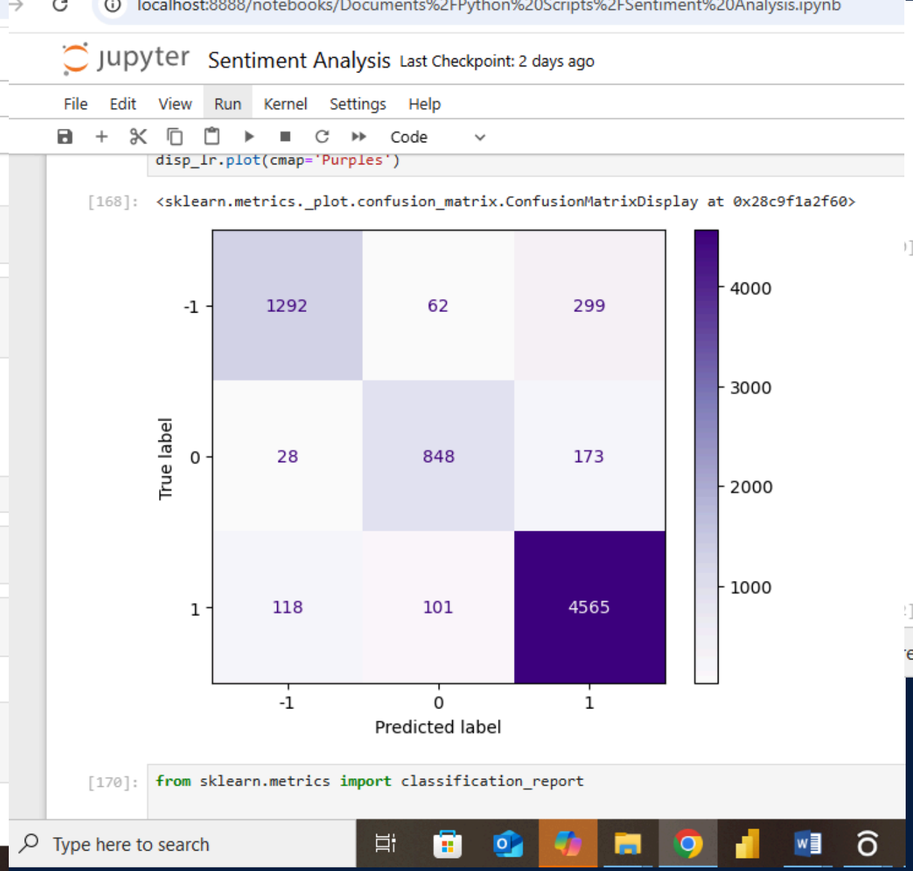
First Modeling Attempt – Naive Bayes

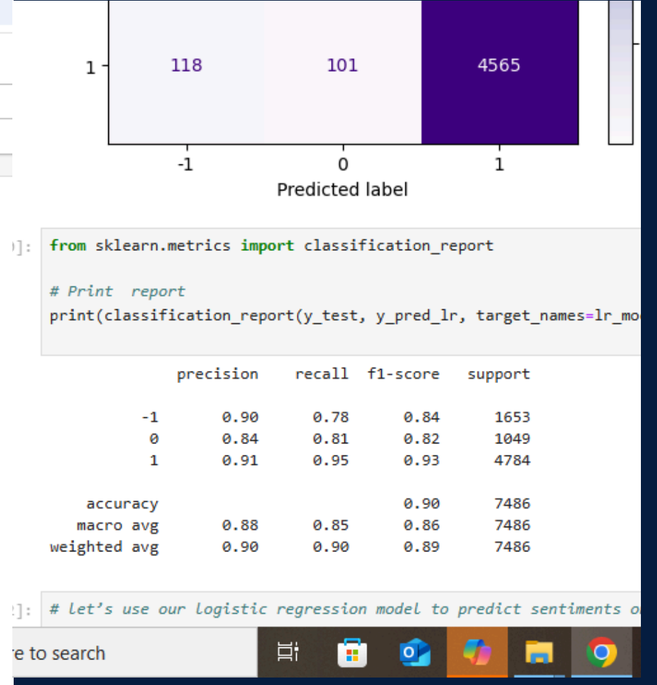
I started with Multinomial Naive Bayes — a classic for text classification. It's fast, easy to implement, and often gives solid baseline results.

However, after evaluating with metrics like precision, recall, and F1-score, the model didn’t perform well enough:

- The Model Accuracy: 74.19 %
- It over-simplified the review patterns.
- Struggled with neutral sentiments (often confusing them as positive or negative).
- The confusion matrix showed too many misclassifications, especially around borderline sentiments.







Upgrading to Logistic Regression

Realizing we needed a more flexible and discriminative model, we transitioned to Logistic Regression. Unlike Naive Bayes, LR doesn't assume independence between features — giving it an edge when dealing with complex patterns in sentiment.

Why Logistic Regression worked better:

- Handled overlapping sentiments better.
- Gave us a clearer decision boundary between sentiment classes.
- Significantly improved F1-score, especially for positive and negative classes.

Making It Visual – Insights from the Data

Overall Sentiment Distribution

localhost:8888/notebooks/Documents%2FPython%20Scripts%2FSentiment%20Analysis.ipynb

jupyter Sentiment Analysis Last Checkpoint: 2 days ago

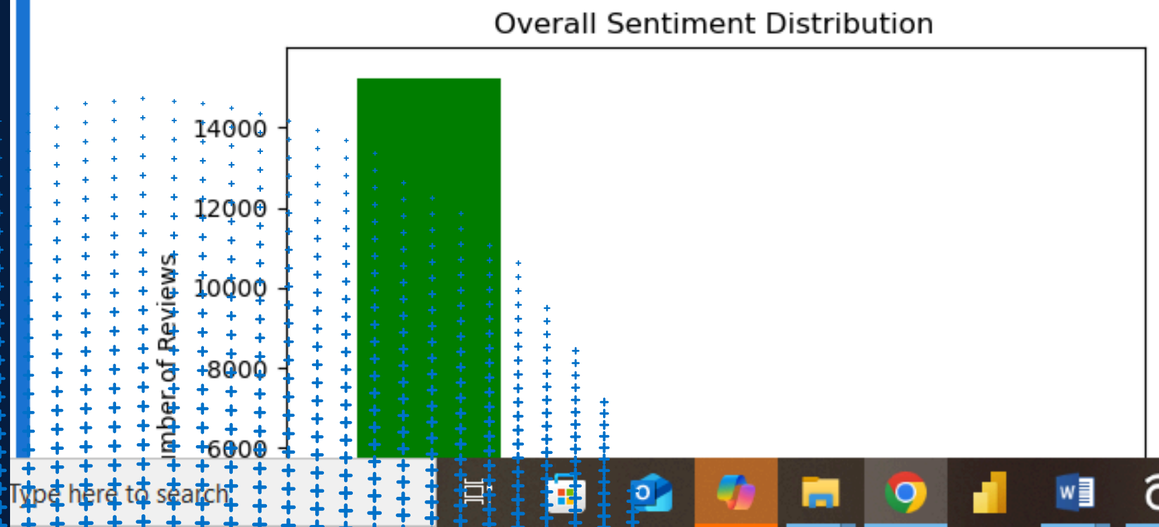
File Edit View Run Kernel Settings Help

Code

```
merged_df = pd.merge(reviews_df, apps_df, on='App')
```

[202]:
```
#Plotting Sentiment Distribution

import matplotlib.pyplot as plt
```

[206]:
```
merged_df['Sentiment'].value_counts().plot(kind='bar', color=['green', 'orange', 'red'])

plt.title("Overall Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Reviews")
plt.xticks(rotation=0)
plt.show()
```



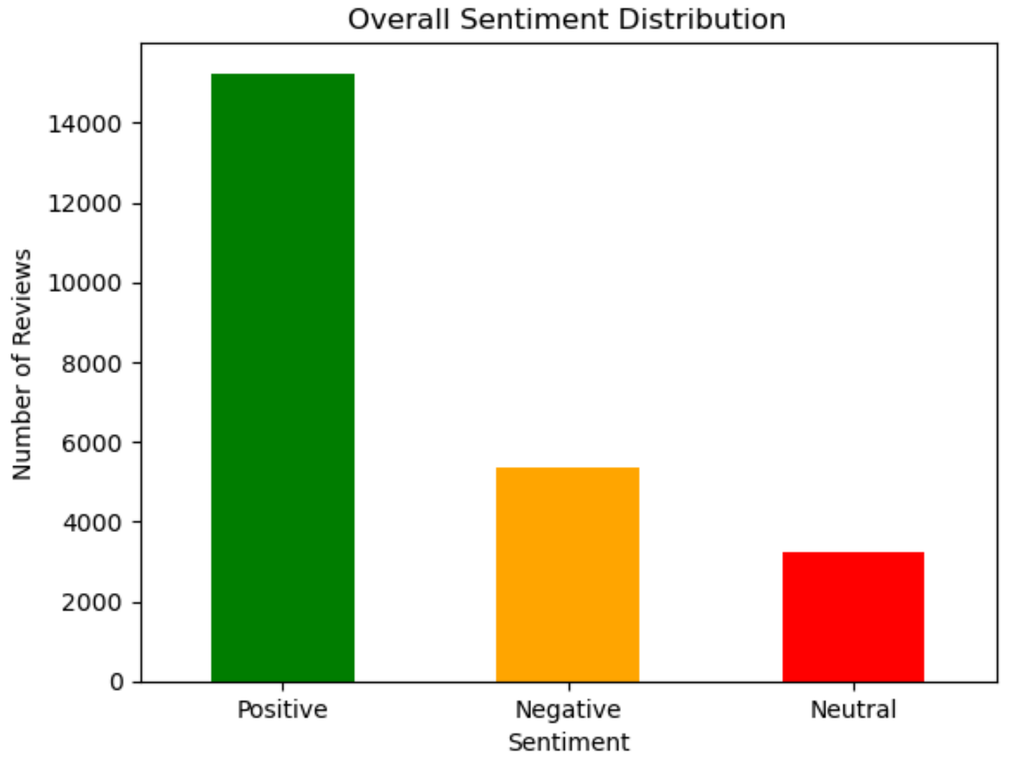
Type here to search

localhost:8888/notebooks/Documents%2FPython%20Scripts%2FSentiment%20Analysis.ipynb

upyter Sentiment Analysis Last Checkpoint: 2 days ago

Edit View Run Kernel Settings Help

Code



11]:
```
import seaborn as sns
sns.set(style="whitegrid")
```

ere to search

Sentiment by App Type (Free vs Paid)

- Free apps get more attention — naturally because more people use them.
- Even though free apps have a lot of positives, they also get a fair share of negatives. Probably because when people don't pay, they tend to be more critical.
- Paid apps are used less, but people who pay seem to be mostly satisfied.

dit View Run Kernel Settings Help

Code

Positive Negative Neutral

Sentiment

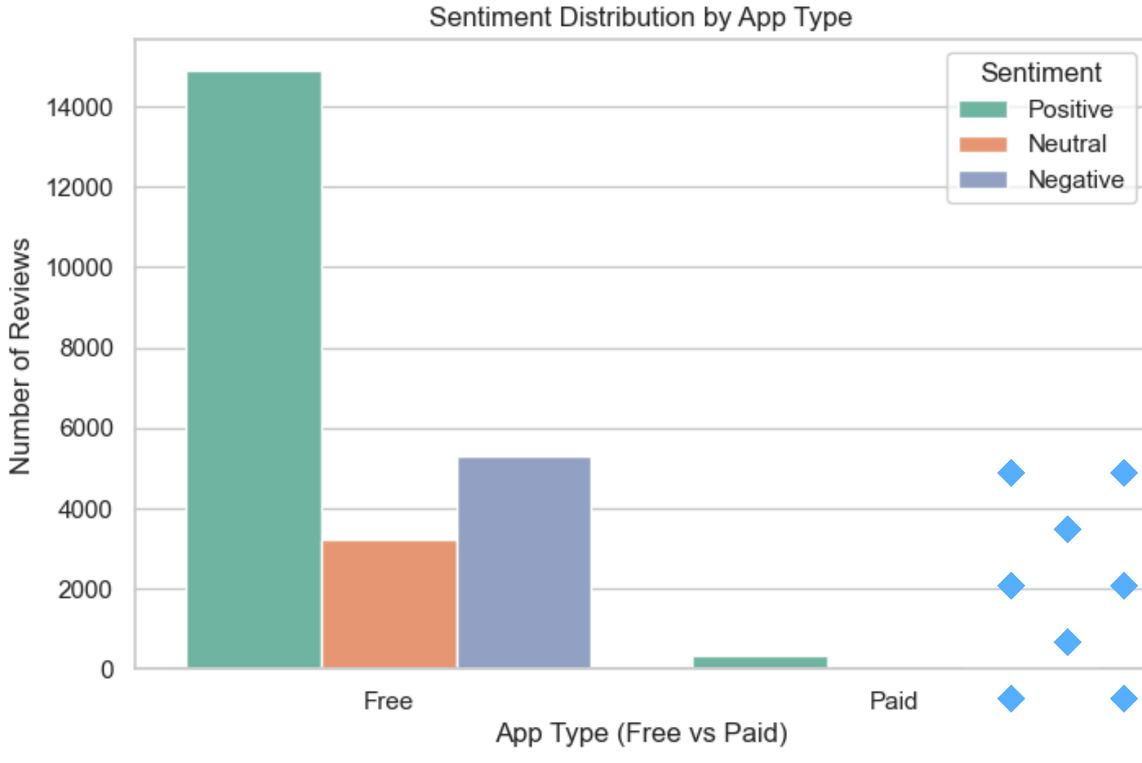
```
]: import seaborn as sns
sns.set(style="whitegrid")

plt.figure(figsize=(8, 5))
sns.countplot(data=merged_df, x='Type', hue='Sentiment', palette='Set2')

plt.title('Sentiment Distribution by App Type')
plt.xlabel('App Type (Free vs Paid)')
plt.ylabel('Number of Reviews')
plt.legend(title='Sentiment')
plt.show()
```

ocalhost:8888/notebooks/Documents%2FPython%20Scripts%2FSentiment%20Analysis.ipynb

yter Sentiment Analysis Last Checkpoint: 2 days ago

View Run Kernel Settings Help

Code



Sentiment by App Category

- Categories like Games, Tools, and Health & Fitness had the highest engagement.
- Where negative sentiment is spiking (like in "GAME")
- Where more user engagement or improvement is needed

localhost:8888/notebooks/Documents%2FPython%20Scripts%2FSentiment%20Analysis.ipynb

oyter Sentiment Analysis Last Checkpoint: 2 days ago

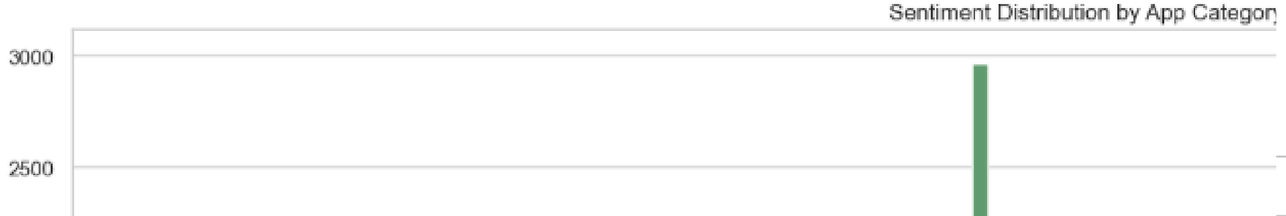
t View Run Kernel Settings Help

Code

Free Paid

App Type (Free vs Paid)

```
#sentiment by app category
category_sentiment = merged_df.groupby(['Category', 'Sentiment']).size().reset_index(name='Re

plt.figure(figsize=(16, 8))
sns.barplot(x='Category', y='Review_Count', hue='Sentiment', data=category_sentiment)
plt.xticks(rotation=90)
plt.title('Sentiment Distribution by App Category')
plt.ylabel('Number of Reviews')
plt.xlabel('App Category')
plt.tight_layout()
plt.show()
```

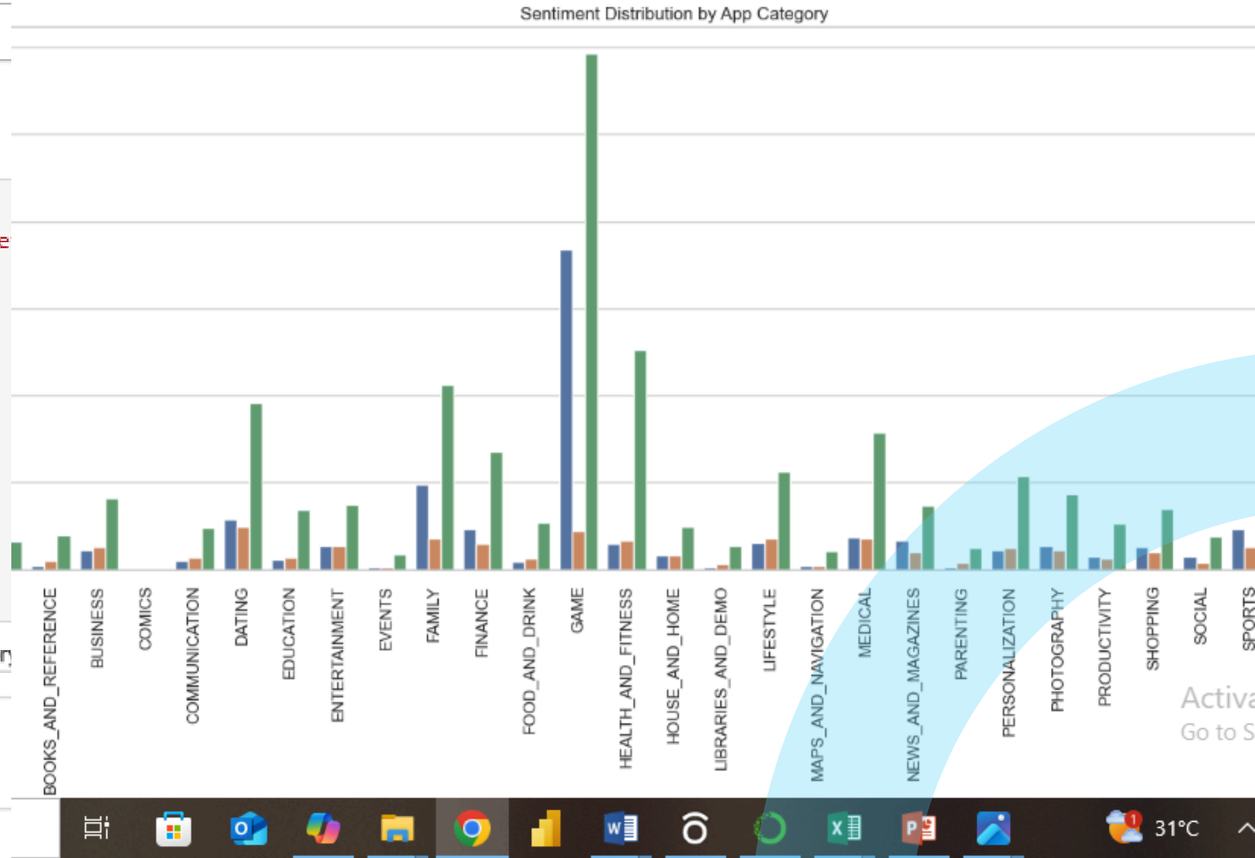


ks/Documents%2FPython%20Scripts%2FSentiment%20Analysis.ipynb

Analysis Last Checkpoint: 2 days ago

Settings Help

Code JupyterLab



✅ Project Milestones Achieved

- ✓ Built sentiment model (Logistic Regression)
- ✓ NLP Preprocessing (TF-IDF)
- ✓ Compared models
- ✓ Visualized key insights
- ✓ Extracted business-level interpretations

Thank's For Watching

Presented by: Ajayi Oluwaseyi

More and Full Analysis on Github: https://github.com/Psychizzy

This project helped me understand how machine learning and NLP can turn feedback into real insight. Thank you for watching